

Practical Research Handoff for Partial Khmer Riel Banknote Detection and Identification

Executive finding

The strongest evidence does **not** point to a single magic model. Instead, the most transferable pattern is a **multi-cue pipeline**: first detect that a visible banknote fragment is present, then decide whether there is enough evidence to name the denomination, and if not, return **unknown / low confidence** instead of forcing a class. The closest direct analogue to your use case is a 2025 mobile system for **mixed and partly covered foreign currency** that used a YOLOv11 detector and explicitly evaluated overlapping, folded, and partially occluded notes; YOLOv11 beat YOLOv8, SSD, and RetinaNet in that study and was deployed as both an on-device TensorFlow Lite model and an optional server-side refinement path. A second highly relevant line is the older banknote-specific literature on **component-based local features**, where systems succeeded specifically because they matched denomination-specific **local patches** rather than requiring the whole note to be visible. A third line comes from generic occlusion literature: visible-region labeling, overlap-aware losses, and instance segmentation architectures materially help when objects occlude each other. ¹

For Khmer Riel specifically, the reviewed sources did **not** surface a peer-reviewed Khmer-Riel-specific detector or recognizer comparable to the USD/EUR/KRW/JOD systems in the literature. The closest directly surfaced Khmer-Riel-specific artifact was a small public Roboflow Universe project with **552 images**. The official National Bank of Cambodia gallery also shows why your task definition matters: there are multiple active denominations and, for several values, multiple note issues or series still documented in circulation, including a **200,000 riel** note issued in **October 2024** and multiple historical/current versions of 100,000, 50,000, 20,000, 10,000, 5,000, 2,000, 1,000, 500, 200, and 100 riels. In practice, this means you should expect to build a custom dataset and decide explicitly whether the label space is “denomination only” or “denomination + series.” ²

If I were handing this off to an engineering team, I would **not** start with a full-note crop classifier, and I would **not** start with amodal segmentation. The most evidence-backed first prototype is a **small visible-region detector** plus a **second-stage verifier** trained on partial crops, with an explicit **unknown** path and optional **video-frame aggregation**. Move to segmentation only if your first real-data error analysis shows that missed detections come mainly from **merged overlapping instances**, not from weak denomination evidence. That sequencing is supported by the direct banknote studies, the mobile deployment literature, and the overlap/occlusion literature together. ³

Existing solutions and what actually transfers

Hasanuzzaman, Yang, and Tian’s component-based SURF system is still one of the clearest demonstrations of why local cues matter for partial banknotes. Their goal was assistive banknote recognition for blind users. The system used manually selected class-specific **reference regions** on each note, extracted SURF features, and accepted a class only if both **point-level** and **region-level** tests passed.

The paper explicitly argues that some note regions carry more class-specific information than others and that a component-based model is less sensitive to viewpoint changes and partial occlusion than a global whole-note model. They evaluated on a deliberately difficult USD dataset with occlusion, cluttered background, rotation, scale, illumination, and viewpoint changes, and reported **100% recognition** on 140 images. What transfers to Khmer Riel is the idea, not the exact implementation: if only a corner or strip is visible, denomination-specific **local motifs** may be more reliable than an end-to-end whole-note classifier. What probably breaks is multi-instance separation in fanned stacks; this system was a recognizer, not a detector/segmenter for many overlapping notes at once. It also depends on having enough stable keypoints in the visible fragment, so motion blur, low light, or very plain note areas can hurt it badly. 4

The Euro Banknote Recognition System for Blind People is a classic two-stage pipeline. It used a **modified Viola-Jones** style detector to find notes, then **SURF** to recognize their value on a Raspberry Pi-like portable system. Reported performance was roughly **84% detection** and **97.5% recognition**. This is useful mainly as a cautionary example: banknote systems often look excellent at the recognition stage only because they first assume a usable note candidate has already been isolated. For your fanned Khmer Riel stack scenario, that isolating-the-note step is the hard part. So the transferable lesson is not “use Viola-Jones,” but rather “separate candidate finding from denomination verification,” because the failure modes differ. On overlapping Khmer Riel notes, the detector stage in such a pipeline is the first thing likely to collapse. 5

Sanchez’s smartphone-video banknote recognizers add another useful design idea: they do not try to classify every frame. The 2015 conference version and the 2019 journal version use **foreground segmentation** on live smartphone video, then compute SURF descriptors only when the foreground has sufficient area and enough keypoints. The 2019 abstraction explicitly mentions **PBAS** foreground segmentation, smartphone videos in real-life scenes, and a **98%** recognition result. The 2015 paper also says the method is robust to clutter, scale change, rotation, illumination, partial occlusion, and a substantial range of 3D viewpoint changes. What transfers well is the **evidence gate**: if a visible fragment is too weak, do not classify it yet. This maps directly onto your “unknown instead of guessing” requirement. What probably breaks for overlapped Khmer Riel stacks is that PBAS-like foreground separation sees the whole hand-held fan as one moving foreground object; it does not naturally separate note instances inside the blob. 6

Carlos Costa’s multiview Euro banknote system and GitHub demo are unusually relevant to your hardest cases. The associated paper/repo states that the system can recognize **multiple banknotes** at different perspectives and scales, in cluttered environments, under varying lighting, and that it can handle banknotes that are **partially visible, folded, wrinkled, worn**, and even **several overlapping banknotes in the same test image**. The pipeline uses preprocessing, local feature detection/description/matching, then computes note contours via **homography**, and validates them with **shape analysis**. The repo reports that all Euro banknotes in **80 test images** were recognized. This is one of the strongest concrete demonstrations that overlapping banknotes can be separated without a neural segmenter if there are enough distinctive local features and a strong geometric verification stage. For Khmer Riel, this suggests a promising **verifier** or fallback module: after a detector proposes a visible fragment, a template/keypoint/homography check could confirm denomination-specific local structure. But as a primary recognizer on a phone/browser, this route is risky because keypoint density collapses on tiny slivers, motion blur, poor light, or repetitive patterns, and engineering cost rises quickly as you add many denominations and series. 7

The “**deep feature-based three-stage detection**” line of work is less directly usable, but it contributes an important banknote-specific trick: explicit **banknote geometry filters**. As summarized in the MBDM review, Faster R-CNN-based banknote detection pipelines used CNN feature extraction plus region proposals, then filtered false positives by **width-to-height ratio**, by the expected **size range** for coins/bills, and by keeping only the highest-scoring duplicate box for one object. That kind of geometry-aware verification is likely useful for Khmer Riel too, especially because notes are planar rectangles with known aspect-ratio families. The limitation is obvious: these filters assume you are seeing a sufficiently complete object. Once only a corner or narrow strip is visible, full-note aspect-ratio assumptions stop being reliable. ⁸

MBDM, the Multinational Banknote Detecting Model, is one of the most relevant modern detection papers, even though it does not focus on extreme partial visibility. Its target problem was smartphone-based **multinational** banknote and coin detection. The authors built a YOLOv3-like custom CNN with **69 layers**, used **four prediction scales** down to **104 × 104** feature maps to help small objects, and improved performance with **mosaic augmentation** that mixed multiple currencies into one training image. The model used **26 classes**, was trained/tested on KRW, USD, EUR, and JOD data, and the paper reports **precision 0.8396, recall 0.9334, F1 0.8840** for the best mosaic setup, plus embedded inference at **17.45 fps** on Jetson TX2. What transfers well is the emphasis on multi-scale prediction, meticulous augmentation, and the acknowledgement that smartphone banknote imagery contains varied backgrounds and mixed classes. What probably breaks on fanned Khmer Riel stacks is that MBDM still assumes classical object detection supervision and does not add any explicit “insufficient visible evidence” state; it is also not built to output masks for separating tightly interleaved visible regions. ⁹

The **2025 YOLOv11 mixed and partly covered foreign currency paper** is the single closest published analogue to your deployment scenario. Its problem statement is almost exactly yours, except that it covered mixed foreign banknotes and coins rather than Khmer Riel. The paper explicitly says it targets **mixed and partly covered** objects, compares **YOLOv8, YOLOv11, SSD, and RetinaNet**, and reports that YOLOv11 was best with **precision 0.951, recall 0.935, mAP@0.5 0.965, and mAP@0.5:0.95 0.906**. It further reports a mobile implementation in **Kotlin** on a **Google Pixel 4XL**, using **TensorFlow Lite** for offline on-device inference and a **FastAPI** backend as an optional high-accuracy path. In real-world trials, the system was reported to correctly detect and identify **93.67%** of partly covered or deformed currency. What transfers directly is that a modern one-stage detector can be competitive even under overlap and deformation, and that a **lightweight mobile model plus optional heavy fallback** is a practical architecture. What remains uncertain is its behavior on the truly hard edge cases you care about: extremely tiny strips, edge-cropped note corners, and cases where the visible region is not denomination-informative. The paper does not describe an explicit “unknown” class or verification stage. ¹⁰

BankNote-Net from Microsoft is not a detector, but it is highly reusable. The project released **24,816** banknote-image embeddings spanning **17 currencies and 112 denominations**, learned with **supervised contrastive learning** and a **MobileNetV2** architecture. Microsoft says a variation of this model is deployed in the public **Seeing AI** app. The repo also includes a path for training a shallow classifier directly from embeddings for currencies in the dataset, and a second path for training on **custom images** from a new currency using the pre-trained encoder. This is particularly interesting for your “unknown instead of guessing” requirement: metric-style embeddings are naturally usable with **distance thresholds** or nearest-neighbor verification, which is often easier to reject than forcing a softmax over known classes. The catch is that BankNote-Net is a **crop classifier / encoder**, not a multi-instance detector, and the custom-image path expects **224 × 224 square** crops. So it is best viewed as a second-stage **denomination verifier** or few-shot adaptation tool, not as the front-end instance finder for fanned notes. ¹¹

Commercial and government examples reinforce the same pattern, even when the methods are undisclosed. Microsoft’s Seeing AI has publicly shipped currency recognition, and Microsoft’s BankNote-Net paper says a variation of its encoder is deployed in the app. The U.S. Treasury’s **Cash Assist** app uses a **continuous scan** flow rather than requiring a captured still image. Google’s official Lookout documentation is even more revealing in the opposite direction: its currency mode asks the user to point the camera at **one banknote at a time**, which strongly suggests that many production systems avoid solving the multi-note overlap problem directly. Cash Reader’s official site and app-store listing go further and **claim** support for over 100 currencies, offline inference, and recognition from small visible portions, but because those are product claims rather than published methods, they should be treated as operational hints rather than scientific evidence. The transferable UX lesson is clear: production apps often use live video, guidance, and conservative rules rather than blind single-frame classification. ¹²

Lessons from occlusion, overlap, and dense-object research

A recurring banknote-specific idea is that **local denomination cues beat whole-note assumptions when visibility is poor**. Hasanuzzaman’s component-based SURF system explicitly says some note regions are more class-specific and more stable than the note as a whole; MBDM’s review also mentions classical approaches that use **fast radial symmetry** or gradients around the **numerical part** of a banknote. Costa’s overlapping-Euro system then adds homography and shape verification on top of local matches. This pattern is more important than the particular algorithms: successful partial-view banknote systems usually rely on **pieces of evidence**—numerals, portraits, seals, borders, or distinctive local print motifs—rather than asking a model to infer denomination from a global full-note view that may not exist. ¹³

The generic occlusion literature strongly supports annotating and modeling the **visible evidence**, not just the imagined full object. CrowdHuman, which was built specifically for severe overlap, provides **visible-region boxes** as well as full-body boxes. OR-CNN improves crowded pedestrian detection with an **aggregation loss** and **part occlusion-aware RoI pooling**. BCNet goes further and explicitly separates **occluder** and **occludee** layers so the system can distinguish true object contours from occlusion boundaries, showing large gains in heavy occlusion cases. End-to-end amodal segmentation models and related datasets such as COCOA push this even further by predicting both visible and invisible parts. The direct implication for banknotes is that **visible-region supervision should be your default first annotation policy**. If you later find that downstream use truly needs estimated full-note extent, that can be layered on as amodal metadata or a future amodal model, but it is not the first thing to optimize. ¹⁴

Dense-object and retail benchmarks are also more transferable than they may look at first glance. **D2S** was designed for instance-aware segmentation in supermarket scenes and highlights limited training data, dense packing, varied lighting, rotations, and the value of **pixel-precise labels that can be reused for artificial data augmentation**. **SKU-110K** focuses on many visually similar objects in close proximity. These are not banknotes, but your “fanned stack” problem is structurally similar: many planar, similar-looking rectangles packed tightly together, often with only partial boundaries visible. The transfer is mostly about labeling philosophy and augmentation strategy, not banknote appearance. ¹⁵

Small-object literature is relevant, but only in a specific sense. If a note fills much of the frame but only a thin corner is visible, your problem is **insufficient semantic evidence**, not merely “small object detection.” Still, tools like **SAHI** are useful when notes or note fragments become small because the phone is farther away, the frame is high-resolution, or you later want browser-based inference on larger images. SAHI reports sizable AP gains on small-object benchmarks by slicing images at inference and fine-tuning time. AI-

TOD and AI-TOD-R further underline that tiny objects are often marginalized by standard assignment strategies. The important caveat is that slicing can recover **spatial resolution**, but it cannot create denomination evidence that is genuinely absent from a plain sliver of paper. ¹⁶

Video is not just a UX convenience here; it is a modeling opportunity. ByteTrack’s core insight is that **low-confidence detections often correspond to real but occluded objects**, and that associating almost every detection box instead of discarding low-score boxes can materially improve tracking continuity. Ultralytics exposes ByteTrack and BoT-SORT directly in its tracking mode, and the banknote-specific video systems for blind users already showed years ago that continuous camera motion plus evidence gating works well in practice. For your scenario, a brief hand movement that reveals different strips of different notes over 5–15 frames could be the difference between “unknown” and a confident denomination, even with the same front-end detector. ¹⁷

Comparing the main approach families for this use case

YOLO-style object detectors are the most practical first family to test. They are fast, well-supported, and directly backed by the two most relevant banknote detection papers here: MBDM and the 2025 YOLOv11 mixed/partly-covered system. They only need box labels, and current Ultralytics export supports **ONNX, TensorRT, CoreML, TensorFlow Lite, TF.js, and NCNN**, which makes phone and browser deployment much easier than with most research repos. Their main weakness in your scenario is not raw speed; it is that overlapping visible regions can get merged or suppressed, and class logits will still try to choose a denomination even when the visible fragment is uninformative unless you add an explicit rejection mechanism. In other words, YOLO is a great **finder**, but it should not be trusted as the sole **namer** for barely visible note fragments. ¹⁸

YOLO segmentation models are the natural next step if overlap separation proves to be the dominant failure mode. Ultralytics’ segmentation task outputs per-instance masks in addition to boxes and classes, and it exports through the same deployment surface as the detection models. That makes it one of the few segmentation options that is still realistic for edge deployment. The trade-off is annotation cost and post-processing complexity. If notes in a fan are mostly separable by visible rectangles, masks may be overkill; if their visible regions interleave tightly around the fingers or edges, masks are often worth the effort because they produce cleaner crops for a second-stage verifier. ¹⁹

RT-DETR and DETR-style detectors deserve serious, but not first-round, consideration. DETR’s main conceptual advantage is that detection is a **set prediction** problem, which removes handcrafted anchor logic and non-maximum suppression. RT-DETR adapts this idea to real-time settings with an **efficient hybrid encoder, IoU-aware query selection**, and adjustable decoder depth; its own paper explicitly frames RT-DETR as a response to the speed/accuracy issues introduced by NMS in YOLO-like pipelines. That matters for your use case because adjacent overlapping notes can be similar and close together, which is exactly where aggressive NMS can hurt recall. The downside is data and engineering appetite: plain DETR historically converged slowly and had issues around small-feature resolution that later variants addressed, and even RT-DETR is generally a harder sell than small YOLOs for low-latency browser or mid-range-phone deployment. It is an excellent second family to benchmark once you have a good dataset, especially if detector NMS is visibly deleting adjacent notes. ²⁰

Mask R-CNN and related instance-segmentation families remain strong if the core problem is “separate several overlapping visible note pieces,” not “run at 30+ fps on mobile.” Their strengths are mature training

recipes, explicit instance masks, and strong interpretability during debugging. TorchVision also documents ONNX export for Mask R-CNN, albeit for **fixed batch size and fixed input size**, which is workable but not as deployment-friendly as the YOLO toolchain. The weakness is simply efficiency: for a phone/browser target, Mask R-CNN is much more plausible as a server-side fallback or offline research baseline than as a first on-device inference engine. ²¹

Detector plus classifier/verifier pipelines are, in my view, the most evidence-backed family for your exact requirement set. This family includes the old SURF systems, the Euro detection+recognition system, BankNote-Net's encoder-plus-shallow-classifier setup, and modern detector-plus-verifier variations you could build with a MobileNet-style classifier. The detector solves "is there a visible note fragment here?" The verifier answers "is there enough local evidence to name the denomination?" That second question is where you can insert an **unknown** outcome. A MobileNet-based verifier is also a good deployment match because banknote literature already cites MobileNet as attractive under low hardware budgets, and BankNote-Net itself uses MobileNetV2. The main design burden is data: the verifier must be trained on **partial crops**, not only on clean full-note crops, otherwise it will confidently hallucinate denominations from weak fragments. ²²

OCR-assisted denomination recognition is plausible as a verifier, not as a primary detector. The most relevant evidence here is indirect: banknote literature includes methods that explicitly target the **numerical part** of the note, and document-focused Khmer OCR resources do exist. There are also modern deep-learning papers on banknote **serial-number OCR**, showing that banknote text can be recognized under unequal illumination and complex backgrounds. But the surfaced Khmer OCR resources are document-oriented, not banknote-specific, and banknote fonts, anti-counterfeit backgrounds, and partial occlusion are much harder than clean document text. So OCR can help if your crop contains a clear Arabic numeral or denomination string, but it should be an optional **confirmation cue**, not the only cue. ²³

Template, keypoint, and banknote-specific hybrid approaches have a surprisingly high upside for this domain. The strongest handcrafted examples in the sources reviewed are exactly the ones that handle partial visibility, wrinkles, viewpoint variation, and overlap best. Their weakness is not that they are "old"; it is that they depend on enough distinctive local structure being visible, and they are harder to optimize across mobile hardware, browser runtimes, and many denomination/series variants. My recommendation is to think of them as **verification modules** for later phases, especially if you discover that certain Khmer Riel denominations have highly distinctive portraits, buildings, numerals, borders, or emblem regions that are visible even in fan stacks. ²⁴

Training, labeling, and evaluation design

Recommendation based on the evidence: define your **label space** before annotating anything. The National Bank of Cambodia documents multiple denominations and, for several denominations, multiple note issues still shown in circulation. If your product requirement is "tell the total value," then your primary class should probably be **denomination**, with **series/version stored as metadata** for error analysis. If your requirement is "tell the exact note type," then you need separate classes or a hierarchical label space. This choice is not cosmetic: local cues that identify a **2015 100 riel** may not be the same as cues that identify a **1995 100 riel**, even though both share denomination. ²⁵

Recommendation based on both banknote and occlusion evidence: annotate **visible regions** for the front-end detector. Do not start by labeling hallucinated full-note boxes when only a corner is visible. CrowdHuman's

visible-region annotations, OR-CNN's occlusion-aware pooling, and BCNet's occluder/occludee modeling all point in the same direction: the visible evidence is what the model can reliably learn first. For your first prototype, a visible-region **box** is usually enough. If your real failure analysis later shows frequent note merging in fans, then add visible-region **masks** for a subset and benchmark whether segmentation materially improves separation. If you eventually want visible-fraction scores or amodal research, keep optional polygon/quadrilateral metadata or synthetic full-note geometry in the annotation schema, but do not make that the main supervision target on day one. ²⁶

Recommendation on ambiguous slivers: not every visible piece should contribute a denomination loss. The smartphone-video systems only run its matcher when enough area/keypoints exist, and BankNote-Net's embedding setup is naturally amenable to thresholded rejection. I would therefore split training examples into at least three regimes: **identifiable denomination fragments**, **note fragments with insufficient denomination evidence**, and **ignore/background**. The middle regime can be handled as a literal **unknown / note-fragment** class, or by excluding it from the classifier loss and using it only for detector supervision. This is one of the fastest ways to stop hallucinated denomination guesses. ²⁷

Recommendation on data generation: build a mixed real-and-synthetic pipeline. MBDM shows that **mosaic augmentation** improves banknote detection in mixed-currency scenes, and D2S explicitly calls out the utility of pixel-precise labels for **artificial augmentation**. Public banknote datasets for Indian/Thai notes also include folded notes, varied backgrounds, and lighting changes, while the damaged-Indian dataset includes soiled, torn, mutated, holed, and dark/cluttered scenes. For your scenario, the synthetic side should generate fan stacks, overlap order, edge truncation, perspective warps, wrinkles, shadows, blur, glare, color shifts, and occluders such as hands; the real side should deliberately capture actual hand-held stacks under the lighting and phone optics you care about. The banknote literature supports the general lesson that training only on clean full notes is not enough. ²⁸

Recommendation on confidence and "unknown" logic: the cleanest practical path is usually a **two-threshold** decision. Threshold one is applied to the detector: is there enough evidence that this visible patch is a banknote instance at all? Threshold two is applied to the verifier: is there enough denomination-specific evidence to label it? BankNote-Net's metric-style embeddings make this especially attractive because cosine distance or nearest-neighbor margin can be thresholded more naturally than a forced closed-set softmax. If you stay fully neural, use a held-out ambiguity set of thin slivers, edge crops, and cross-denomination look-alikes specifically to calibrate the unknown behavior. That calibration step is an engineering recommendation, but it is strongly motivated by the evidence-gating behavior in the earlier video/SURF systems and by the few-shot/embedding framing of BankNote-Net. ²⁹

Recommendation on evaluation: your main metric should not be standard COCO AP alone, because you already said you care more about **finding partially visible notes** than about perfect boxes. I would keep standard AP for comparability, but add three task-specific views: **visible-note recall** at a lenient IoU for visible-region labels, **denomination accuracy conditional on sufficient evidence**, and **unknown precision** for truly ambiguous fragments. Also stratify results by visible fraction and scene type. CrowdHuman's distinction between visible region and full box gives you the conceptual template for this kind of difficulty bucketing. ³⁰

Prototype architectures and recommended experiments

Single-stage detector only is the cheapest baseline and is worth training first simply because the 2025 YOLOv11 paper and MBDM show that a modern detector can already do a lot under clutter, overlap, and deformations, and because export/deployment to phone or browser is straightforward through ONNX, TFLite, CoreML, TF.js, or NCNN. The quick test is simple: train on visible-region boxes with denomination labels and inspect whether the model already finds most visible notes in fan stacks. The likely failure mode is precisely the one you are worried about: the detector will often assign a denomination to fragments that are not actually informative enough. If you see that happening early, do not try to solve it only with confidence thresholds; add a verifier. ³¹

Detector plus crop verifier is the architecture I would test first in earnest. Use the detector to produce visible fragments; crop a slightly padded patch; then pass it to either a lightweight classifier or an embedding verifier. The evidence for this family comes from multiple directions: the older banknote systems separated candidate localization from recognition, and BankNote-Net shows that a contrastive, MobileNetV2-based encoder can generalize across many currencies and adapt to new ones with custom images. The fast experiment is to compare detector-only against detector-plus-verifier on a dev set of deliberately ambiguous slivers and corner-only crops. If the verifier reduces false denomination guesses without killing recall too badly, you have validated the central product requirement. For phone deployment, this is also attractive because a small detector plus a tiny classifier is often easier to optimize than a single bigger “do-everything” model. ³²

Segmenter plus crop verifier is the next architecture if overlap is your dominant failure mode. Use YOLO-seg for speed-sensitive deployments or Mask R-CNN as a strong, slower baseline. The evidence base here is mostly transferable rather than banknote-specific: BCNet and related overlap-aware segmentation work show why explicit instance masks help in heavy occlusion, and Ultralytics’ segmentation/export path makes YOLO segmentation one of the few realistic edge-friendly options. The fast experiment is not to relabel everything with masks; it is to annotate a smaller subset of your hardest fan-stack images with visible-region masks and measure whether separation improves enough to justify the annotation cost. If most failures disappear with masks, segmentation is probably worth the labeling budget. If not, the real bottleneck is likely denomination ambiguity, not overlap separation. ³³

Detector plus OCR/template verification is best treated as a targeted add-on, not as your default architecture. This is the family to reach for if you discover on Khmer Riel that certain denominations are strongly identifiable from a visible Arabic numeral, Khmer denomination text, border motif, or portrait patch. The direct evidence is that banknote literature repeatedly uses denomination numerals or local reference regions, and the Costa-style feature matching system shows that geometric verification can survive overlap and wrinkles if enough distinctive local structure is present. The fast experiment here is selective: manually tag a few candidate cue regions on canonical Khmer Riel images and test whether those cues remain visible often enough in your fan-stack data to justify a verifier branch. If they do not, skip this family for now. ³⁴

Detector plus video tracking and confidence aggregation is one of the most underused but best-supported ideas for your scenario. The literature already gives you both ingredients: continuous-scan banknote UX from assistive apps and evidence recovery from low-confidence boxes via ByteTrack. The fast test is straightforward: take short handheld clips instead of isolated stills, run your detector frame by frame, track instances, and aggregate verifier evidence over time per track. If a note is unknown in frame 1

because only a blank strip is visible, but the user shifts the stack and frame 5 reveals a denomination numeral, the track-level decision can lift confidence without hallucinating. This architecture is especially attractive for phone capture because it improves robustness without requiring much more annotation than the still-image detector/verifier pipeline. ¹⁷

Lightweight on-device model with optional server fallback is the most production-realistic architecture if you care about browser and phone simultaneously. The 2025 YOLOv11 paper already validated the pattern: a TensorFlow Lite model for offline low-latency inference plus an optional backend for refinement. On the tooling side, current YOLO exports cover ONNX, TFLite, CoreML, TF.js, and NCNN, while ONNX Runtime Web supports browser inference through standard JS imports plus experimental **WebGPU** and **WebNN** backends. My recommendation is to keep the edge path intentionally conservative—detector plus verifier with unknown allowed—and reserve the server path for heavy segmentation, higher-resolution reprocessing, or uncertain tracks only. That gives you a good user experience without pretending the edge model can always identify every sliver. ³⁵

The approaches I would test **first** are therefore: a visible-region YOLO-style detector; the same detector plus a partial-crop verifier; and a short-video aggregation version of that pair. The approaches I would treat as **later** or **riskier** are: pure full-note crop classification, OCR-only denomination recognition, and amodal/occlusion-aware segmentation from the start. The first group is directly supported by the closest banknote papers and the deployability tooling. The second group may eventually be useful, but it carries higher label cost, more implementation complexity, or weaker direct evidence for your exact “barely visible banknote” setting. ³⁶

A practical **two-week prototype** should try to prove four things and nothing more ambitious. First, can a small visible-region detector recover most visible notes in your real fan-stack scenes? Second, can a second-stage verifier materially reduce wrong denomination guesses on partial crops? Third, does short video aggregation recover notes that stay unknown in single frames? Fourth, do masks actually help on your hardest overlap cases, or are boxes enough for the MVP? If, after two weeks, you can answer those four questions with real Khmer Riel data, you will have removed most of the decisive project risk. The biggest remaining unknowns after that are likely to be label-space policy across note series, the most reliable local Khmer Riel cues for verification, and the percentage of real-world fragments that should honestly remain **unknown** rather than classified. ³⁷

¹ ³ ¹⁰ ¹⁸ ³¹ ³⁵ ³⁶ <https://www.mdpi.com/2673-2688/6/10/241>
<https://www.mdpi.com/2673-2688/6/10/241>

² <https://universe.roboflow.com/khmer-riel-classification-computer-vision/cambodia-currency-project>
<https://universe.roboflow.com/khmer-riel-classification-computer-vision/cambodia-currency-project>

⁴ ¹³ ²⁴ <https://pmc.ncbi.nlm.nih.gov/articles/PMC4270050/>
<https://pmc.ncbi.nlm.nih.gov/articles/PMC4270050/>

⁵ ²² ³² <https://doaj.org/article/a32b5b3fbc19403dbfb46d75c74c2545>
<https://doaj.org/article/a32b5b3fbc19403dbfb46d75c74c2545>

⁶ <https://www.dbpia.co.kr/journal/detail?nodeId=T13873155>
<https://www.dbpia.co.kr/journal/detail?nodeId=T13873155>

7 <https://github.com/carlosmccosta/Currency-Recognition>

<https://github.com/carlosmccosta/Currency-Recognition>

8 9 23 28 34 <https://www.mdpi.com/2227-7390/11/6/1392>

<https://www.mdpi.com/2227-7390/11/6/1392>

11 29 <https://github.com/microsoft/banknote-net>

<https://github.com/microsoft/banknote-net>

12 <https://blogs.microsoft.com/accessibility/seeing-ai-new-features/>

<https://blogs.microsoft.com/accessibility/seeing-ai-new-features/>

14 26 30 <https://www.crowdhuman.org/>

<https://www.crowdhuman.org/>

15 <https://arxiv.org/abs/1804.08292>

<https://arxiv.org/abs/1804.08292>

16 <https://arxiv.org/abs/2202.06934>

<https://arxiv.org/abs/2202.06934>

17 <https://arxiv.org/abs/2110.06864>

<https://arxiv.org/abs/2110.06864>

19 <https://docs.ultralytics.com/tasks/segment>

<https://docs.ultralytics.com/tasks/segment>

20 <https://arxiv.org/abs/2005.12872>

<https://arxiv.org/abs/2005.12872>

21 [https://docs.pytorch.org/vision/main/models/generated/](https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.maskrcnn_resnet50_fpn.html)

[torchvision.models.detection.maskrcnn_resnet50_fpn.html](https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.maskrcnn_resnet50_fpn.html)

https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.maskrcnn_resnet50_fpn.html

25 37 https://www.nbc.gov.kh/english/about_the_bank/banknotes_in_circulation.php

https://www.nbc.gov.kh/english/about_the_bank/banknotes_in_circulation.php

27 <https://journal.kci.go.kr/jksci/archive/articlePdf?artiId=ART002474599>

<https://journal.kci.go.kr/jksci/archive/articlePdf?artiId=ART002474599>

33 <https://arxiv.org/abs/2103.12340>

<https://arxiv.org/abs/2103.12340>